NESNE YÖNELİMLİ PROGRAMLAMA-1

Sınıf ve Nesne

__init__ yapıcı metodu ve self parametresi

class __init__ self metot

ogrenci.py

```python
class Ogrenci:
    def __init__(self, ad):
        self.ad = ad

erhan = Ogrenci("Erhan")
print(erhan.ad)  # Erhan
```

Erhan AKAGÜNDÜZ

BU HAFTA

Dersin sonunda neler yapabileceksiniz?

class

Sınıf tanımlamak, ondan nesne üretmek ve doğru isimlendirmek

__init__

Yapıcı metodun ne zaman ve nasıl çalıştığını açıklamak; parametre ve varsayılan değer vermek

self

self'in neyi temsil ettiğini ve unutulunca oluşan hataları tanımak

Metot

Nesnenin durumunu değiştiren metotlar yazmak ve iz sürmek

Geçen hafta "neden?"i konuştuk, bu hafta "nasıl?" sorusuna geçiyoruz. Her satırın perde arkasında ne olduğunu göreceğiz.

GEÇEN HAFTADAN BUGÜNE

Bu kodu kullandık. Şimdi anlama zamanı.

hafta2_hesap.py

```python
class Hesap:                                # 1
    def __init__(self, sahip, bakiye): # 2
        self.sahip = sahip              # 3
        self.bakiye = bakiye

    def para_yatir(self, miktar):
        self.bakiye += miktar           # 4

h = Hesap("Ömer", 1000)                     # 5
```

1 · class tam olarak ne tanımlar?
Bölüm 1 · class

2 · Neden iki alt çizgi? Kim çağırıyor?
Bölüm 2 · __init__

3 · self nereden geliyor?
Bölüm 3 · self

4 · Bakiye nerede saklanıyor?
Bölüm 4 · metotlar ve durum

5 · Bu satır çalışınca ne oluyor?
Bölüm 2 · nesnenin doğuşu

BÖLÜM 1

01 class

Yeni bir tür tanımlamak ve o türden nesneler üretmek.

- en basit sınıf
- nesne üretmek
- isimlendirme
- dışarıdan nitelik eklemek

CLASS · EN BASİT SINIF

İki satırlık bir sınıf

bos_sinif.py

```
class Ogrenci:
    pass

a = Ogrenci()
b = Ogrenci()
print(type(a))
print(a == b)
print(a is b)
print(isinstance(a, Ogrenci))
```

▷ ÇIKTI

```
<class '__main__.Ogrenci'>
False
False
True
```

`class`	“Yeni bir tür tanımlıyorum” anahtar kelimesi
`pass`	“Gövde boş” demek; Python boş blok kabul etmez
`Ogrenci()`	Parantezle çağırınca yeni bir **nesne** üretilir (örnekleme)
`a is b`	**False**: aynı sınıftan, ama iki ayrı nesne

isinstance(nesne, Sinif) “bu nesne bu sınıfın bir örneği mi?” sorusunu yanıtlar.

CLASS · İSİMLENDİRME (PEP 8)

İsmine bakınca ne olduğunu anla

Ne?	Yazım stili	Örnek
Sınıf	PascalCase	Ogrenci, BankaHesabi
Nesne / değişken	snake_case	ogrenci1, banka_hesabi
Metot	snake_case (fiil)	para_yatir, not_ekle
Nitelik	snake_case (isim)	bakiye, dogum_yili
Özel metot	__çift_alt_çizgi__	__init__, __str__

Sınıf adı **tekil isim**: **Ogrenci**, Ogrenciler değil. Bir nesne tek bir öğrencidir.

Python 3 Türkçe harfleri kabul eder ama **ç, ş, ğ** yerine **c, s, g** kullanmak taşınabilirlik için daha güvenli.

CLASS · SORUN

Nitelikleri dışarıdan eklemek: tehlikeli

disaridan.py

```
class Ogrenci:
    pass
a = Ogrenci()
a.ad = "Ada"
a.numara = 101
b = Ogrenci()
b.ad = "Efe"          # numara unutuldu!
print(a.ad, a.numara)
try: print(b.ad, b.numara)
except AttributeError as e: print("Hata:", e)
```

▷ ÇIKTI

```
Ada 101
Hata: 'Ogrenci' object has no attribute 'numara'
```

Neden kötü?

- ⚠ Aynı sınıftan nesnelerin **farklı nitelikleri** olabiliyor.
- ⚠ Bir niteliği unutmak ancak **kullanılınca** fark edilir.
- ⚠ Sınıfa bakan biri bir öğrencinin neleri olduğunu **göremiyor**.

ϟ İstediğimiz: her nesne **doğduğu anda** gerekli niteliklerle donatılsın. İşte **__init__**'in işi bu.

BÖLÜM 2

02

__init__

Yapıcı metot: her nesne doğarken otomatik çalışan kurulum kodu.

ne zaman çalışır? | nesnenin doğuşu | parametreler | varsayılan değerler | doğrulama

Bebek doğar doğmaz kimliği verilir

__init__, bir nesne oluşturulduğu anda Python tarafından **otomatik** çağrılan ve nesnenin **başlangıç durumunu** kuran özel metottur.

1 · İSTEK

`Ogrenci("Ada", 101)`

"Bana yeni bir öğrenci üret."

→

2 · DOĞUM

`__init__ çalışır`

ad ve numara nesneye yazılır.

→

3 · HAZIR

`ad="Ada", numara=101`

Kullanıma hazır nesne.

init = initialize (başlatmak). Türkçe kaynaklarda "yapıcı metot" veya "kurucu" denir.

Çift alt çizgili (**dunder**) isimler Python'a özeldir: **biz çağırmayız, Python çağırır**.

Kimse çağırmadı, ama çalıştı

init_ilk.py

```python
class Ogrenci:
    def __init__(self):
        print("Yeni öğrenci oluşturuldu!")

a = Ogrenci()
b = Ogrenci()
print("Program bitti.")
```

▷ ÇIKTI

```
Yeni öğrenci oluşturuldu!
Yeni öğrenci oluşturuldu!
Program bitti.
```

Dikkat edin

- ✓ Kodda **__init__()** diye bir çağrı **yok**.
- ✓ Mesaj **iki kez** yazıldı: her nesne için bir kez.
- ✓ **Ogrenci()** satırı çalıştığı **anda** tetiklendi.

Kural: **Sinif()** yazmak = yeni nesne + o nesne için **__init__** çalışır.

__INIT__ · PERDE ARKASI

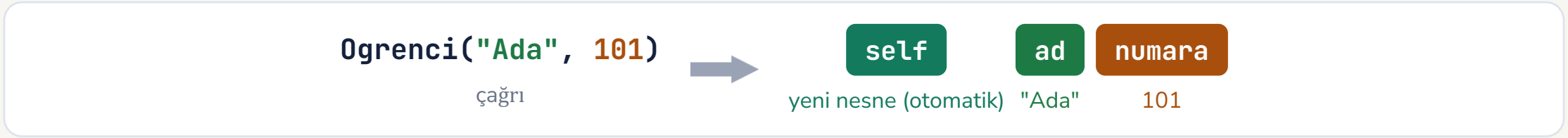
Bir satırda dört adım

```
h = Ogrenci("Ada", 101)
```

1	Python bellekte **boş** bir Ogrenci nesnesi oluşturur.	`nesne → { }`
2	__init__ çağrılır; yeni nesne **self** olarak verilir.	`__init__(nesne, "Ada", 101)`
3	**self.ad = ad** gibi satırlar niteliği nesneye yazar.	`nesne → {ad: "Ada", numara: 101}`
4	__init__ biter; hazır nesne **h** ismine bağlanır.	`h → {ad: "Ada", numara: 101}`

Not: 1. adımı aslında **__new__** adlı başka bir özel metot yapar; günlük kodda ona neredeyse hiç dokunmayız.

Parantezdeki değerler __init__'e gider




```python
class Ogrenci:
    def __init__(self, ad, numara):
        self.ad = ad
        self.numara = numara

a = Ogrenci("Ada", 101)
b = Ogrenci("Efe", 102)
print(a.ad, a.numara)
print(b.ad, b.numara)
try: c = Ogrenci("Nil")
except TypeError as e: print("Hata:", e)
```

▷ ÇIKTI

```
Ada 101
Efe 102
Hata: Ogrenci.__init__() missing 1 required
positional argument: 'numara'
```

Artık numarasız öğrenci **oluşturulamaz**. 7. slayttaki sorun çözüldü.

Varsayılanlar ve sabit başlangıçlar

varsayilan.py

```
class Ogrenci:
    def __init__(self, ad, numara, bolum="Yazılım"):
        self.ad = ad
        self.numara = numara
        self.bolum = bolum
        self.notlar = []     # her zaman boş başlar

a = Ogrenci("Ada", 101)
b = Ogrenci("Efe", 102, "Elektrik")
print(a.bolum, a.notlar)
print(b.bolum, b.notlar)
```

▷ ÇIKTI

```
Yazılım []
Elektrik []
```

Varsayılan parametre

bolum="Yazılım": verilmezse bu değer kullanılır.

Parametresiz nitelik

self.notlar = []: dışarıdan gelmiyor; her öğrenci boş listeyle başlıyor.

⚠ Varsayılanlı parametreler **sona** yazılır. Liste gibi değişebilir varsayılanlara dikkat: 34. slayt.

__INIT__ · DOĞRULAMA

Hatalı nesne hiç doğmasın

dogrulama.py

```
class Urun:
    def __init__(self, ad, fiyat):
        if fiyat < 0:
            raise ValueError("Fiyat negatif olamaz")
        self.ad = ad
        self.fiyat = fiyat
k = Urun("Kalem", 15)
print(k.ad, k.fiyat)
try: h = Urun("Hatalı", -5)
except ValueError as e: print("Hata:", e)
```

▷ ÇIKTI

```
Kalem 15
Hata: Fiyat negatif olamaz
```

Kapıda kontrol

__init__ nesnenin **giriş kapısıdır**. Kurallar burada kontrol edilirse geçersiz bir nesne hiç oluşmaz.

raise bir hata fırlatır ve __init__ yarıda kesilir; **h** değişkeni hiç oluşmaz.

Başka hangi kurallar?

Boş ad olamaz · not 0–100 arası · yaş pozitif · e-posta "@" içermeli

Hata yönetimi ve özel hata sınıfları: **13. hafta**.

__init__ doldurur, bir şey döndürmez

init_return.py

```python
class Deneme:
    def __init__(self):
        return 42

try:
    d = Deneme()
except TypeError as e:
    print("Hata:", e)
```

▷ ÇIKTI

```
Hata: __init__() should return None, not 'int'
```

	__init__	Normal metot
Kim çağırır?	Python, otomatik	Biz: nesne.metot()
Kaç kez?	Nesne başına 1 kez	İstediğimiz kadar
Ne döndürür?	Her zaman None	İstediğimiz değeri
Adı	Sabit: __init__	Biz seçeriz
Görevi	Başlangıç durumu	Davranış

Nesneyi **Ogrenci(...)** çağrısı döndürür; __init__ sadece içini doldurur.

__init__ bir kez çalışır: bunu unutma

dikdortgen.py

```
class Dikdortgen:
    def __init__(self, en, boy):
        self.en = en
        self.boy = boy
        self.alan = en * boy
        self.cevre = 2 * (en + boy)

d = Dikdortgen(4, 3)
print(d.alan, d.cevre)
d.en = 10
print(d.alan)          # güncellenmedi!
```

▷ ÇIKTI

```
12 14
12
```

Ne oldu?

En 10 oldu ama alan hâlâ **12**. Çünkü alan, nesne doğarken **bir kez** hesaplanıp kaydedildi.

Daha iyisi: metot

```
def alan(self):
    return self.en * self.boy
```

Her çağrıda **güncel** değerle hesaplanır: **d.alan()**

Parantezsiz **d.alan** yazımını korumanın yolu: **@property** (5. hafta).

BÖLÜM 3

03

self

Metodun içinden “bu nesne” demenin yolu.

self kimdir? | perde arkası | id ile kanıt | unutursak ne olur? | self.ad ve ad

SELF · KAVRAM

self, Türkçedeki “ben” gibidir

self, bir metodun **hangi nesne için** çalıştığını tutan ilk parametredir. Metodu çağıran nesne neyse, self odur.

GÜNLÜK HAYATTA

Ada “**Benim** adım Ada.”

Erhan “**Benim** adım Erhan.”

Aynı kelime (“ben”), konuşana göre farklı kişiyi gösterir.

PYTHON’DA

```
a.selamla()  → self = a
b.selamla()  → self = b
```

Aynı metot kodu, çağıran nesneye göre farklı veriyle çalışır.

SELF · PERDE ARKASI

Noktanın solundaki nesne, ilk parametre olur

```
s.soyle("Merhaba")  =  Selam.soyle(s, "Merhaba")
```

```
class Selam:
    def __init__(self, ad):
        self.ad = ad
    def soyle(self, mesaj):
        return f"{self.ad}: {mesaj}"

s = Selam("Ada")
print(s.soyle("Merhaba"))
print(Selam.soyle(s, "Merhaba"))
```

▷ ÇIKTI

```
Ada: Merhaba
Ada: Merhaba
```

İki yazım, aynı iş

Python **s.soyle("Merhaba")** satırını arka planda **Selam.soyle(s, "Merhaba")** çağrısına çevirir.

Yani **self** sihir değildir: sadece otomatik doldurulan ilk argümandır.

Günlük kodda her zaman **s.soyle(...)** yazılır; ikinci yazım sadece anlamak için.

id() ile kanıtlayalım: self o nesnenin ta kendisi

self_id.py

```
class Kutu:
    def kimim(self):
        print("self :", id(self))
k1 = Kutu()
k2 = Kutu()
print("k1   :", id(k1))
k1.kimim()
print("k2   :", id(k2))
k2.kimim()
```

▷ ÇIKTI

```
k1   : 140046808217456
self : 140046808217456
k2   : 140046806505168
self : 140046806505168
```

Sayılar birebir aynı

id() bir nesnenin bellekteki kimlik numarasını verir. k1.kimim() içinde self, k1 ile **aynı nesne**; k2 için de öyle.

⚠ Numaralar her çalıştırmada değişir; önemli olan **eşleşmeleri**.

SELF · HATA 1

Parametre listesinde self'i unutmak

hata1.py

```
class Ogrenci:
    def selamla():      # self yok!
        print("Merhaba")
a = Ogrenci()
try:
    a.selamla()
except TypeError as e:
    print("Hata:", e)
```

▷ ÇIKTI

```
Hata: Ogrenci.selamla() takes 0 positional arguments
but 1 was given
```

"0 bekliyordum, 1 geldi"

Biz parantez içine bir şey yazmadık. Peki o "1 argüman" kim?

Cevap: Python'un otomatik gönderdiği **a** nesnesi. Ama metot onu alacak bir yer (self) açmamış.

Düzeltme

```
def selamla(self):
```

SELF · HATA 2

Atamada “self.” önekini unutmak

hata2.py

```
class Ogrenci:
    def __init__(self, ad):
        ad = ad          # self. unutuldu
a = Ogrenci("Ada")
try:
    print(a.ad)
except AttributeError as e:
    print("Hata:", e)
```

▷ ÇIKTI

```
Hata: 'Ogrenci' object has no attribute 'ad'
```

Sinsi hata: program hemen çökmez

ad = ad sadece yerel değişkeni kendisine atar. Nesneye hiçbir şey yazılmaz.

__init__ bitince yerel **ad** yok olur; hata çok sonra, okurken çıkar.

Düzeltme

```
self.ad = ad
```

SELF · 1. HAFTAYLA BAĞLANTI

self.ad ile ad aynı şey değil

```
self.ad = ad
```

sol: nesnenin niteliği (kalıcı)
sağ: metodun yerel parametresi (geçici)

__init__'İN YEREL ALANI

`self` `ad = "Ada"`

Metot bitince bu isimler **silinir** (1. hafta: yerel kapsam).

self.

NESNE (BELLEKTE)

`ad = "Ada"`

Nesne yaşadığı sürece **kalır**; diğer metotlar **self.ad** ile okur.

Bir değeri metotlar arasında taşımak istiyorsan onu **self** üzerine yaz. Yoksa metotla birlikte kaybolur.

self bir anahtar kelime değil, bir gelenektir

bu.py

```
class Nokta:
    def __init__(bu, x, y):
        bu.x = x
        bu.y = y

n = Nokta(3, 4)
print(n.x, n.y)
```

▷ ÇIKTI 3 4

Çalışıyor, çünkü...

Python için önemli olan ismin değil, **sıranın**: ilk parametre her zaman nesnedir. Ona ne ad verirsen ver.

⚠ Ama asla böyle yazmayın

PEP 8 ve tüm Python topluluğu **self** kullanır. Başka isim, kodunuzu okuyan herkesi şaşırtır.

Karşılaştırma: Java ve C#'ta bunun adı **this**'tir ve gizlidir; Python ise onu açıkça yazdırır.

BÖLÜM 4

04

Metotlar ve nesne durumu

Nesneler veri taşır; metotlar bu veriyi okur ve değiştirir.

metot metodu çağırır | durum değişimi | __dict__ | print(nesne)

Metotlar self üzerinden birbirini çağırır

ogrenci.py

```python
class Ogrenci:
    def __init__(self, ad):
        self.ad = ad
        self.notlar = []
    def not_ekle(self, n):
        self.notlar.append(n)
    def ortalama(self):
        return sum(self.notlar) / len(self.notlar)
    def durum(self):
        if self.ortalama() >= 50:
            return "Geçti"
        return "Kaldı"

o = Ogrenci("Ada")
o.not_ekle(40); o.not_ekle(75)
print(o.ortalama(), o.durum())
```

▷ ÇIKTI 57.5 Geçti

Çağrı zinciri

`o.durum()`

↓ `self.ortalama()`

↓ `self.notlar → [40, 75]`

⚠ Sınıf içinden **ortalama()** yazmak NameError verir; doğrusu **self.ortalama()**.

Nesne hatırlar: her metot durumu değiştirir

araba.py

```python
class Araba:
    def __init__(self, marka):
        self.marka = marka
        self.hiz = 0
    def hizlan(self, miktar):
        self.hiz += miktar
    def fren(self):
        self.hiz = max(0, self.hiz - 30)

a = Araba("Togg")
a.hizlan(50); print(a.hiz)
a.hizlan(40); print(a.hiz)
a.fren();     print(a.hiz)
a.fren(); a.fren(); a.fren(); print(a.hiz)
```

Çağrı	hiz	Çıktı
Araba("Togg")	0	–
hizlan(50)	50	50
hizlan(40)	90	90
fren()	60	60
fren() × 3	30 → 0 → 0	0

Nitelikler nesnenin **durumu**dur (state). Metotlar bu durumu kurallara göre değiştirir: **max(0, ...)** hızın eksiye düşmesini engelliyor.

Her nesnenin içinde bir sözlük var

dict.py

```
class Ogrenci:
    def __init__(self, ad, numara):
        self.ad = ad
        self.numara = numara
a = Ogrenci("Ada", 101)
b = Ogrenci("Erhan", 102)
b.numara = 999
print(a.__dict__)
print(b.__dict__)
```

▷ ÇIKTI

```
{'ad': 'Ada', 'numara': 101}
{'ad': 'Erhan', 'numara': 999}
```

a

```
ad: "Ada"
numara: 101
```

b

```
ad: "Erhan"
numara: 999
```

1. haftayla bağlantı

self.ad = ad aslında nesnenin sözlüğüne **"ad"** anahtarını ekler. 2. haftada sözlükten sınıfa geçtik; şimdi görüyoruz ki sınıf, sözlüğü **düzenli ve kurallı** hâle getiriyor.

Her nesnenin **kendi** sözlüğü var; b'yi değiştirmek a'yı etkilemedi.

print(nesne) neden bu kadar çirkin?

```
# sorun
class Ogrenci:
    def __init__(self, ad):
        self.ad = ad

a = Ogrenci("Ada")
print(a)
```

▷ ÇIKTI

```
<__main__.Ogrenci object at 0x7f5f302f42f0>
```

Mavi: hangi sınıfın nesnesi

Turuncu: bellek adresi (değişir)

```
# şimdilik çözüm: kendi metodumuz
    def bilgi(self):
        return f"{self.ad} ({self.numara})"

a = Ogrenci("Ada", 101)
print(a.bilgi())   # Ada (101)
```

6. haftada: __str__

Özel bir metot yazınca **print(a)** doğrudan **Ada (101)** yazacak. __init__ gibi o da Python'un otomatik çağırdığı bir dunder metot.

SÖZLÜK

Bu haftanın terimleri

Terim	İngilizce	Anlamı	Örnek
Sınıf	class	Nesnelerin planı, yeni bir tür	Ogrenci
Nesne, örnek	object, instance	Sınıftan üretilen somut varlık	a
Örnekleme	instantiation	Sınıftan nesne üretmek	Ogrenci("Ada")
Nitelik	attribute	Nesnenin taşıdığı veri	a.ad
Metot	method	Nesneye bağlı fonksiyon	a.not_ekle(90)
Yapıcı	initializer	Başlangıç durumunu kuran özel metot	__init__
self	self	Metodu çağıran nesnenin kendisi	self.ad
Durum	state	Niteliklerin o anki değerleri	hiz = 60

UYGULAMA · MİNİ PROJE

Kontenjanlı ders kaydı

ders.py

```python
class Ders:
    def __init__(self, ad, kontenjan):
        self.ad = ad
        self.kontenjan = kontenjan
        self.ogrenciler = []

    def kayit_ol(self, ogrenci):
        if len(self.ogrenciler) >= self.kontenjan:
            print(f"{ogrenci}: kontenjan dolu!")
            return
        self.ogrenciler.append(ogrenci)
        print(f"{ogrenci} kaydoldu.")

    def bos_yer(self):
        return self.kontenjan - len(self.ogrenciler)
```

```python
# kullanım
oop = Ders("OOP", 2)
oop.kayit_ol("Ada")
oop.kayit_ol("Efe")
oop.kayit_ol("Nil")
print(oop.ogrenciler, "| boş:", oop.bos_yer())
```

▷ ÇIKTI

```
Ada kaydoldu.
Efe kaydoldu.
Nil: kontenjan dolu!
['Ada', 'Efe'] | boş: 0
```

Bugünkü her şey burada: **__init__**, **self**, liste nitelik, kural koyan metot.

UYGULAMA · İZ SÜRME

oop nesnesinin hayatı, adım adım

#	Çağrı	self.ogrenciler	Kontrol	Sonuç
1	Ders("OOP", 2)	[]	–	__init__ nesneyi kurdu
2	kayit_ol("Ada")	['Ada']	0 < 2	Ada kaydoldu.
3	kayit_ol("Efe")	['Ada', 'Efe']	1 < 2	Efe kaydoldu.
4	kayit_ol("Nil")	['Ada', 'Efe']	2 ≥ 2	dolu, return
5	bos_yer()	['Ada', 'Efe']	2 − 2	0

```
print(oop.__dict__)
{'ad': 'OOP', 'kontenjan': 2, 'ogrenciler': ['Ada', 'Efe']}
```

Her çağrıda **self** aynı nesne (oop). Liste metotlar arasında **kalıcı**, çünkü self üzerinde duruyor.

KONTROL LİSTESİ

Bir sınıf yazdıktan sonra sor

- Sınıf adı **PascalCase** ve tekil mi?
- Her nitelik **self.** ile mi atanıyor?
- Metot içinden nitelik ve metotlara **self.** ile mi ulaşılıyor?
- Geçersiz veri **__init__**'te reddediliyor mu?
- **__init__** zorunlu bilgileri parametre olarak alıyor mu?
- Her metodun ilk parametresi **self** mi?
- Liste gibi **değişebilir varsayılan** parametre var mı? (Olmamalı)
- Türetilen değerler (alan, ortalama) **metotla** mı hesaplanıyor?

KENDİNİ SINA

Tahmin et: ekranda ne görünür?

Soru 1 • self.x mi, x mi?

```
class A:
    def __init__(self):
        self.x = 1
        x = 5

a = A()
print(a.x)
```

1

x = 5 yerel bir değişken; nesneye yazılmadı. a.x hala 1.

Soru 2 • Paylaşılan sepet?

```
class Sepet:
    def __init__(self, urunler=[]):
        self.urunler = urunler

s1 = Sepet()
s2 = Sepet()
s1.urunler.append("elma")
print(s2.urunler)
```

['elma']

Varsayılan [ ] bir kez oluşur ve iki sepet **aynı listeyi** paylaşır! Çözüm: **urunler=None**.

DİKKAT

En sık yapılan 6 hata ve mesajları

```
def selamla():
```

self unutuldu → **TypeError**: takes 0 positional arguments but 1 was given

```
ad = ad
```

self. unutuldu → nitelik oluşmaz; okurken **AttributeError**

```
def _init_(self, x):
```

Tek alt çizgi → Python tanımaz: **A() takes no arguments**

```
a = Ogrenci
```

Parantez yok → **a** nesne değil, sınıfın kendisi olur.

```
return ortalama()
```

Sınıf içinde → **NameError**. Doğrusu: **self.ortalama()**

```
def __init__(self, l=[]):
```

Tüm nesneler **aynı listeyi** paylaşır. Varsayılanı **None** yapın.

ÖZET

Bugün öğrendiklerimiz, dört cümlede

`class`

Yeni bir tür tanımlar. **Sinif()** ile ondan nesne üretilir; her nesne ayrıdır.

`__init__`

Nesne doğarken otomatik çalışır, başlangıç durumunu kurar, None döndürür.

`self`

Metodu çağıran nesnedir. **a.m(x)** = **Sinif.m(a, x)**. Kalıcı veri **self.** üzerinde durur.

Metot ve durum

Nitelikler nesnenin durumudur; metotlar onu kurallara göre okur ve değiştirir.

Sınıf bir kez yazılır, self sayesinde binlerce nesne için çalışır.

PRATİK ÖDEV · GELECEK HAFTA

Kendi sınıflarını yaz

1 · Kitap

ad, yazar, sayfa (pozitif olmalı), okunan = 0. Metotlar: **oku(n)**, **kalan()**, **yuzde()**.

2 · Banka hesabı

sahip, bakiye = 0, gecmis = []. Her yatırma ve çekme geçmişe yazılsın; **ozet()** hepsini listelesin.

3 · Hata avı

35. slayttaki 6 hatanın her birini kasıtlı yap, mesajı kaydet, sonra düzelt.

4 · İz sür

Araba: hizlan(70), fren(), hizlan(15), fren(), fren(). Her adımda hiz kaç?

HAFTA 4

Sınıf değişkenleri, örnek değişkenleri

Tüm öğrencilerin **toplam sayısını** kim tutar? Hiçbir nesne tek başına bilemez.

```
class Ogrenci:
    sayac = 0  # ?
```